

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Тульский государственный университет»

Институт прикладной математики и компьютерных наук

Кафедра вычислительной техники

Отчет по лабораторной работе №9

На тему «Тестирование в CI/CD и анализ качества кода»

По дисциплине «Процессы обеспечения качества программного
обеспечения»

Выполнил

_____ ст. гр. 220621: Забродин И. В.

Проверил

_____ ст. п. кафедры ВТ, к. т. н.: Пестин М. С.

Тула 2026

Цель работы

Изучить принципы интеграции процесса тестирования в конвейер непрерывной интеграции и доставки (CI/CD), освоить инструменты статического и динамического анализа кода, а также научиться настраивать автоматическую генерацию отчетов о качестве и покрытии кода тестами.

Ход работы

Описание статического анализа (Задание 1)

Перечень использованных инструментов статического анализа

StyleCop.Analyzers — анализатор стиля кода для C#. Был добавлен в проект Core через NuGet. Позволяет выявлять нарушения стандартов кодирования, такие как неправильные отступы, именование переменных, отсутствие документации и т. д.

Встроенный анализатор .NET (dotnet build с предупреждениями) — использовался для выявления потенциальных ошибок компиляции и предупреждений.

Обоснование выбора: *StyleCop.Analyzers* является стандартным инструментом для проектов на C#, легко интегрируется и не требует отдельного сервера. Встроенный анализатор .NET уже присутствует в SDK и дополняет проверку.

Пример работы анализатора *StyleCop.Analyzers*:

По умолчанию *StyleCop.Analyzers* требует, чтобы публичные методы были задокументированы (Рисунок 1)

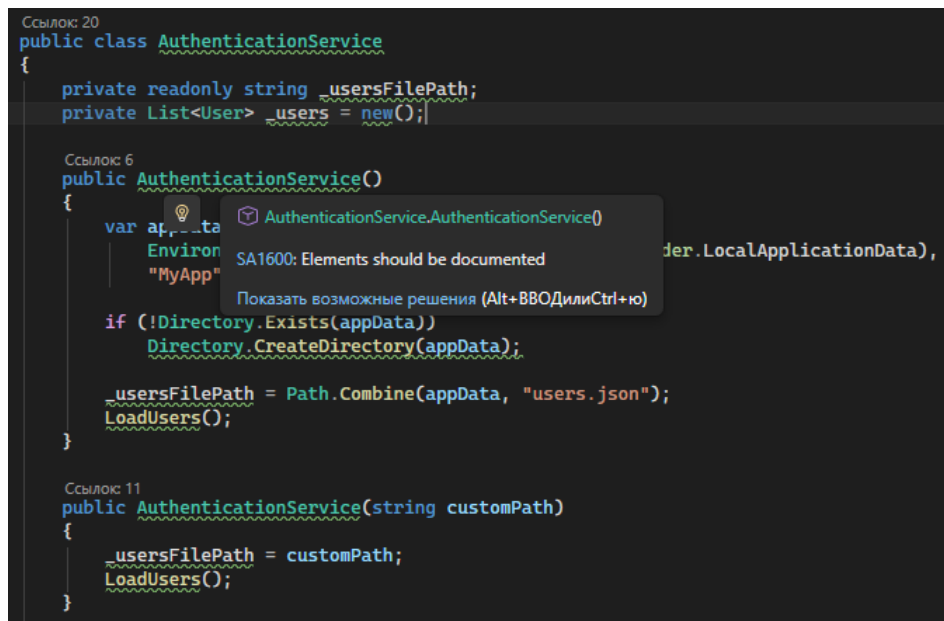


Рисунок 1 – Рекомендация документирования публичного метода (конструктора)

Добавим в .editorconfig строку:

```
dotnet_diagnostic.SA1600.severity = none
```

Теперь анализатор не ругается на не задокументированные методы (Рисунок 2)

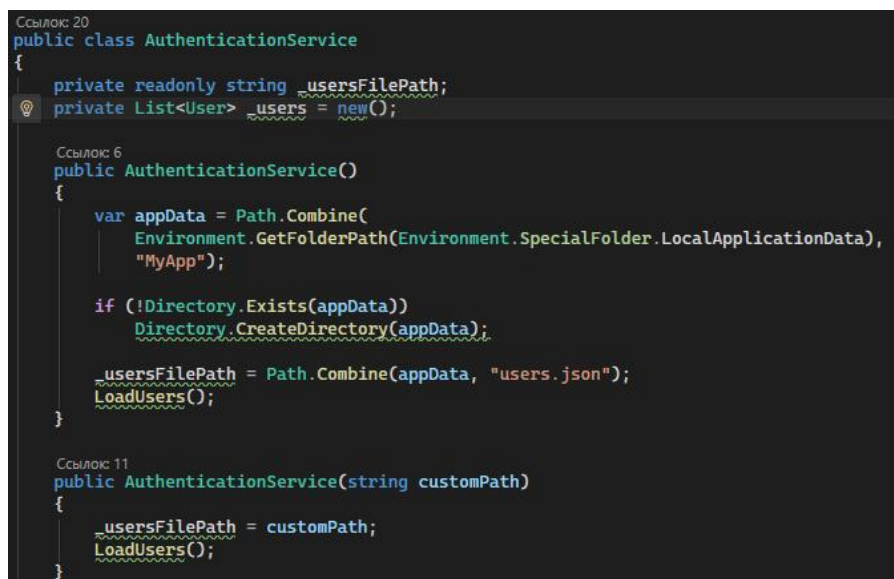


Рисунок 2 – Не задокументированные методы не подчёркиваются

Описание CI/CD пайплайна (Задание 2)

Листинг .github/workflows/ci.yml:

```
name: .NET CI/CD
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: windows-latest
    strategy:
      matrix:
        project: [UnitTests, IntegrationTests]
    steps:
      - uses: actions/checkout@v4

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x
      - name: Restore dependencies
        run: dotnet restore MyApp/MyApp.slnx

      - name: Build
        run: dotnet build MyApp/MyApp.slnx --no-restore --configuration Release

      - name: Check code style (optional)
        run: dotnet format --verify-no-changes --verbosity diagnostic || true

      - name: Run ${ matrix.project } tests with coverage
        run: |
          dotnet test ${ matrix.project }/${ matrix.project }.csproj `
            --configuration Release `
            --no-build `
            --collect:"XPlat Code Coverage" `
            --settings MyApp/coverlet.runsettings `
            --results-directory ./TestResults/${ matrix.project }
      - name: Upload test results
        uses: actions/upload-artifact@v4
        with:
          name: test-results-${ matrix.project }
          path: ./TestResults/${ matrix.project }
          if: always()
  report:
    needs: test
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4

      - name: Download all test results
        uses: actions/download-artifact@v4
        with:
          path: ./TestResults

      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: 8.0.x

      - name: Install ReportGenerator
        run: dotnet tool install -g dotnet-reportgenerator-globaltool

      - name: Generate coverage report
        run: |
          reportgenerator `
            -reports:"./TestResults/**/*.cobertura.xml" `
            -targetdir:"./coveragereport" `
            -reporttypes:Html
      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: ./coveragereport
```

Пояснения:

Пайплайн состоит из двух стадий: test и report.

В стадии test параллельно запускаются модульные и интеграционные тесты на Windows Runner (из-за WinForms).

После тестов результаты сохраняются как артефакты.

В стадии report из всех отчётов о покрытии генерируется единый HTML-отчёт с помощью ReportGenerator и также публикуется как артефакт.

Скриншоты успешного выполнения пайплайна

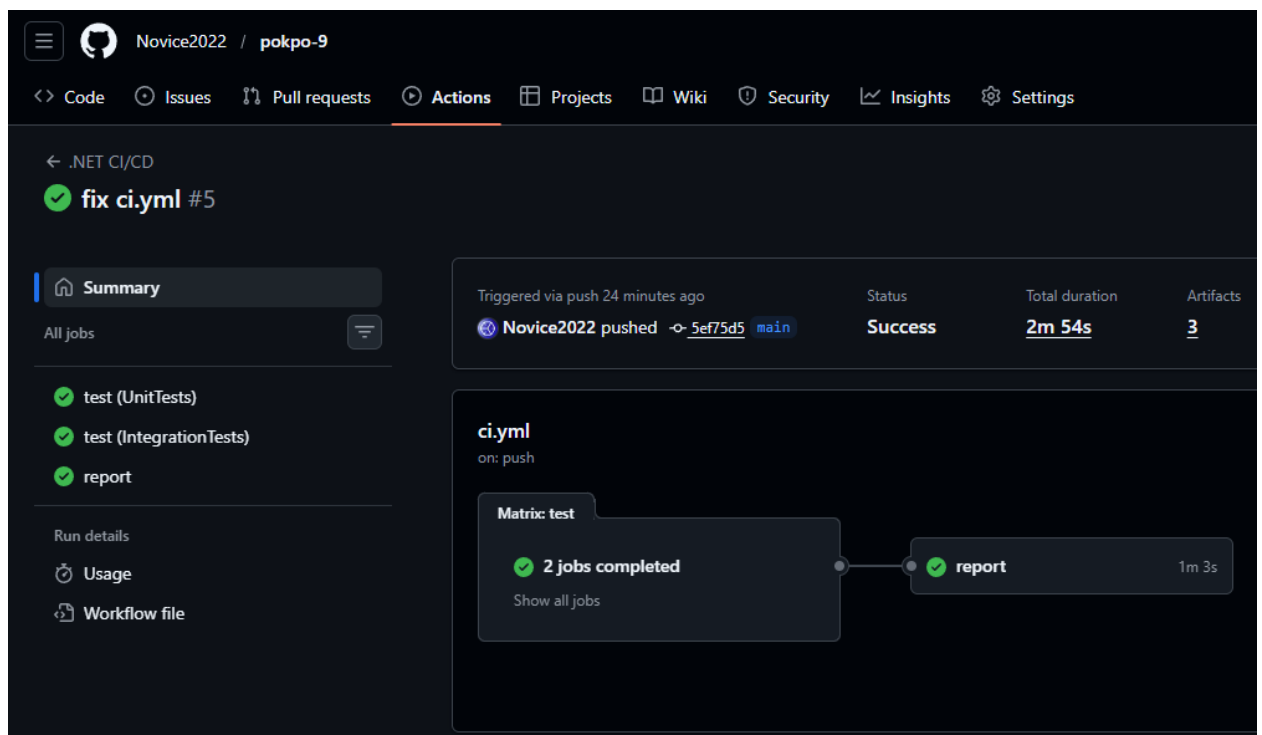


Рисунок 3 – Успешно выполненный pipeline

Описание анализа покрытия (Задание 3)

Отчёт о покрытии кода тестами (Рисунок 4)

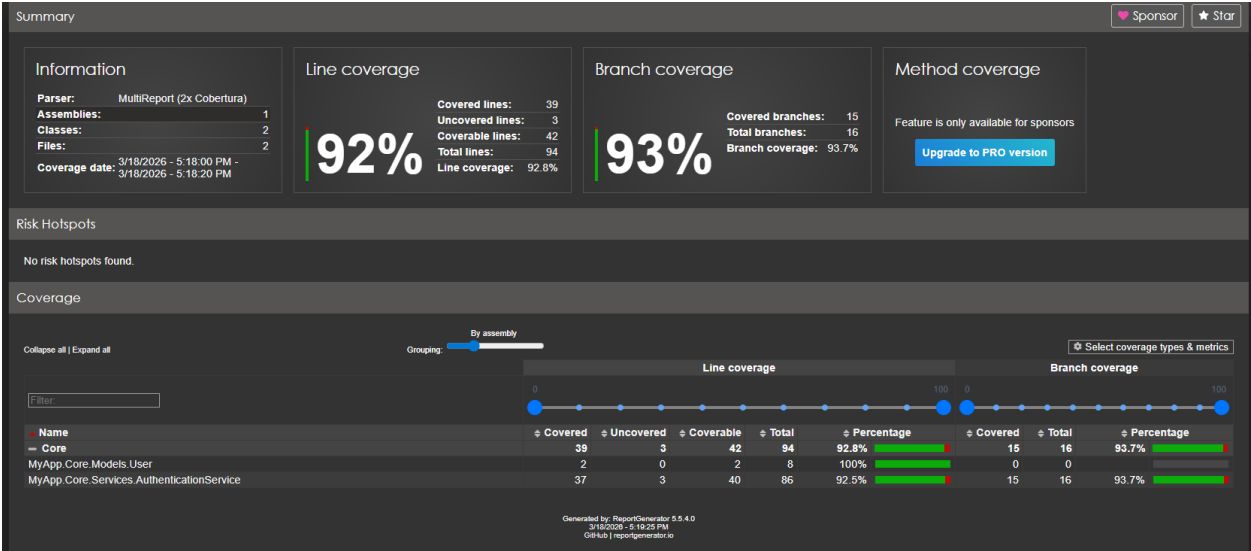


Рисунок 5 – Отчёт о покрытии кода, сгенерированный ReportGenerator.

Не покрытые строки (Рисунок 6)

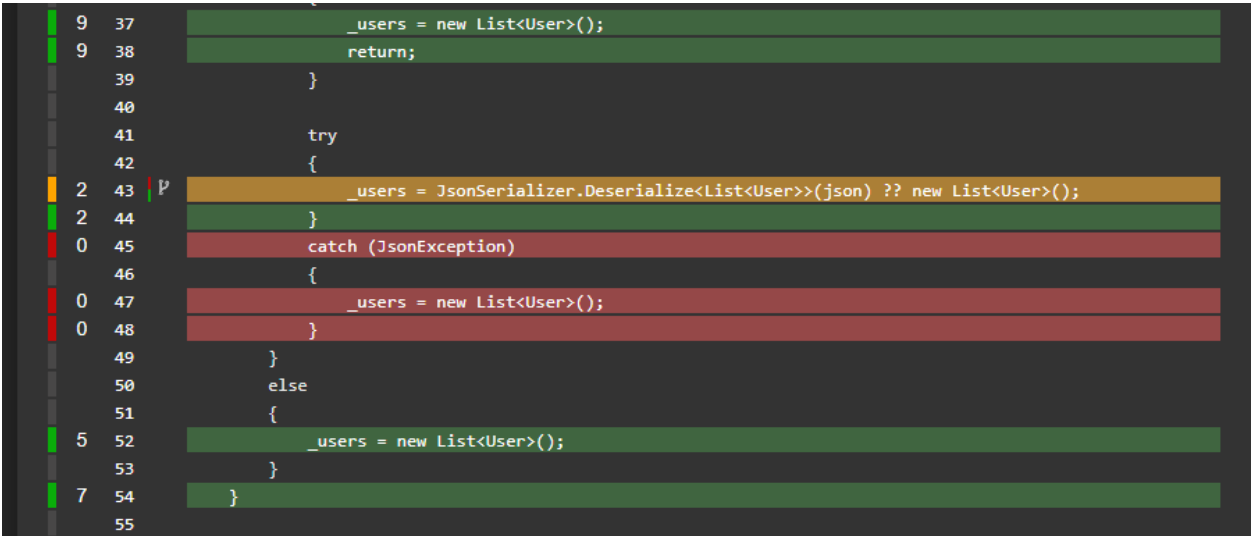


Рисунок 6 – Не покрытые строки

Выводы

В ходе выполнения лабораторной работы был настроен конвейер CI/CD на базе GitHub Actions, включающий статический анализ кода (StyleCop), запуск модульных и интеграционных тестов, а также автоматическую генерацию отчётов о покрытии. Интеграция этих проверок в процесс разработки позволяет:

Обнаруживать ошибки на ранних этапах (до слияния кода).

Автоматически контролировать качество кода и соблюдение стандартов.

Гарантировать, что в основную ветку попадает только протестированный код.

Получать наглядную статистику о покрытии, что помогает улучшать тестирование.

Таким образом, CI/CD повышает надёжность программного продукта и ускоряет разработку за счёт автоматизации рутинных проверок.

Ответы на контрольные вопросы

1) Что такое CI/CD и каковы его основные компоненты?

CI/CD (Continuous Integration/Continuous Delivery) — это практика автоматизации сборки, тестирования и развертывания приложений. Основные компоненты: пайплайн (последовательность задач), стадии (группы задач), задачи (отдельные скрипты), артефакты (результаты выполнения), кэш (для ускорения сборок), раннеры (агенты выполнения).

2) Чем отличаются непрерывная доставка (Continuous Delivery) от непрерывного развертывания (Continuous Deployment)?

Непрерывная доставка подразумевает, что код после автоматического тестирования готов к релизу, но развертывание в production происходит вручную. Непрерывное развертывание — полностью автоматический процесс, при котором каждое успешное изменение сразу попадает в production.

3) Что такое пайплайн (pipeline), стадия (stage) и задача (job)?

Пайплайн — это набор автоматизированных процессов, объединённых в стадии. Стадия — логическая группа задач, выполняемых последовательно (например, тестирование, сборка). Задача — минимальная единица работы внутри стадии (например, запуск конкретных тестов).

4) Какие преимущества дает интеграция автоматического тестирования в CI/CD?

Позволяет быстро обнаруживать ошибки при каждом изменении кода, предотвращает попадание дефектов в основную ветку, сокращает время на ручное тестирование, повышает уверенность в качестве продукта.

5) Чем артефакты (artifacts) отличаются от кэша (cache) в CI/CD?

Артефакты — это результаты сборки или тестирования (например, отчёты, DLL-файлы), которые сохраняются для дальнейшего использования или анализа. Кэш — это временное хранилище зависимостей (NuGet-пакетов,

node_modules) для ускорения последующих сборок, не предназначенное для долгого хранения.

6) Что такое статический анализ кода и какие проблемы он помогает выявить?

Статический анализ — проверка исходного кода без его выполнения. Помогает выявить нарушения стиля, потенциальные логические ошибки, неиспользуемые переменные, дублирование кода, уязвимости, сложные конструкции.

7) Чем статический анализ отличается от динамического (модульного тестирования)?

Статический анализ не требует выполнения программы, он анализирует исходный код или байт-код. Динамический анализ (тестирование) выполняется на работающей программе и проверяет её поведение на конкретных входных данных.

8) Что такое "запах кода" (code smell)? Приведите примеры.

Запах кода — поверхностные признаки, которые могут указывать на более глубокие проблемы в коде (не обязательно ошибки). Примеры: длинные методы, большие классы, дублирование кода, магические числа, чрезмерная вложенность условий.

9) Что такое цикломатическая сложность и почему важно ее контролировать?

Цикломатическая сложность — метрика, измеряющая количество линейно независимых путей в коде. Высокая сложность означает, что код трудно тестировать, понимать и поддерживать. Контроль сложности помогает писать более простой и надёжный код.

10) Какие инструменты статического анализа для вашего языка программирования вы знаете?

Для C#: StyleCop.Analyzers, SonarQube/SonarCloud, ReSharper, Roslyn Analyzers. Для Java: SpotBugs, PMD, Checkstyle. Для JavaScript/TypeScript: ESLint, JSHint. Для PHP: PHPStan, PHP_CodeSniffer.

11) Что такое покрытие кода (code coverage) и какие его виды существуют?

Покрытие кода — метрика, показывающая, какая часть исходного кода выполняется при запуске тестов. Основные виды: покрытие строк (line coverage) и покрытие ветвлений (branch coverage). Также бывает покрытие функций, условий и путей.

12) Какие инструменты для измерения покрытия кода вы знаете?

Для .NET: Coverlet, dotnet-coverage. Для Java: JaCoCo, Cobertura. Для Python: pytest-cov, coverage.py. Для JavaScript: Istanbul, Jest.

13) Является ли 100% покрытие кода гарантией отсутствия ошибок? Почему?

Нет, не является. Покрытие лишь показывает, какие строки были выполнены, но не гарантирует, что тесты проверяют все возможные сценарии и граничные случаи. Можно иметь 100% покрытие, но пропустить логические ошибки или ошибки в интеграции.

14) Для чего используются Allure Reports?

Allure Reports — это фреймворк для создания наглядных, интерактивных отчётов о выполнении тестов. Он агрегирует результаты тестирования, вложения, шаги и метрики в единый удобный для анализа веб-отчёт.

15) Как настроить публикацию отчетов в GitHub Pages или GitLab Pages?

В CI/CD пайплайне нужно сгенерировать статический сайт с отчётом (например, HTML), затем использовать специальное действие (в GitHub Actions — peaceiris/actions-gh-pages) для публикации содержимого папки с

отчётом в ветку gh-pages. В настройках репозитория включить GitHub Pages для этой ветки.

16) Как настроить защиту ветки (branch protection), чтобы требовать успешного прохождения CI-проверок перед слиянием?

В GitHub: Settings → Branches → Add rule → указать ветку (например, main) → включить "Require status checks to pass before merging" → выбрать необходимые проверки из списка (например, test, lint, report) → сохранить.

17) Что такое CodeQL и для чего он используется?

CodeQL — это анализатор безопасности от GitHub, встроенный в GitHub Actions. Он сканирует код на наличие уязвимостей, таких как SQL-инъекции, XSS, небезопасное использование API, и помогает обнаружить их до попадания в production.

18) Как безопасно хранить чувствительные данные (токены, пароли) в CI/CD пайплайнах?

Использовать встроенные механизмы хранения секретов: в GitHub Actions — Secrets (Settings → Secrets and variables → Actions), в GitLab CI/CD — CI/CD Variables. Секреты подставляются в переменные окружения и не отображаются в логах.

19) Что такое allow_failure в GitLab CI/CD и для чего он используется?

Параметр allow_failure позволяет задаче завершиться с ошибкой, но не останавливать весь пайплайн. Используется, например, для необязательных проверок (линтер, экспериментальные тесты), которые не должны блокировать слияние.

20) Как обеспечить быстроедействие CI/CD пайплайнов (ускорение сборки и тестирования)?

Использовать кэширование зависимостей (NuGet, npm), параллельное выполнение задач (матрицы), запускать только нужные тесты (например, по изменённым файлам), использовать более мощные раннеры, минимизировать шаги, не требующиеся на каждом запуске.

Листинг coverlet.runsettings:

```
<?xml version="1.0" encoding="utf-8"?>
<RunSettings>
  <DataCollectionRunSettings>
    <DataCollectors>
      <DataCollector friendlyName="XPlat code coverage">
        <Configuration>
          <Format>cobertura</Format>
          <Exclude>[*Tests]*</Exclude>
          <IncludeTestAssembly>>false</IncludeTestAssembly>
          <!-- Порог общего покрытия строк -->
          <Threshold>70</Threshold>
          <ThresholdType>Line</ThresholdType>
        </Configuration>
      </DataCollector>
    </DataCollectors>
  </DataCollectionRunSettings>
</RunSettings>
```

Листинг.gitignore:

```
## Ignore Visual Studio temporary files, build results, and
## files generated by popular Visual Studio add-ons.

# User-specific files
*.suo
*.user
*.useroscache
*.sln.docstates

# Build results
[Dd]ebug/
[Dd]ebugPublic/
[Rr]elease/
[Rr]eleases/
x64/
x86/
build/
bld/
[Bb]in/
[Oo]bj/
```